StockPilot

Inventory management shipped through a full DevSecOps pipeline, from git push to a monitored, autoscaling EKS cluster

DevOps Final Capstone · Session 21 · SidantxDev · October 2026



What I built: a warehouse inventory dashboard

The problem

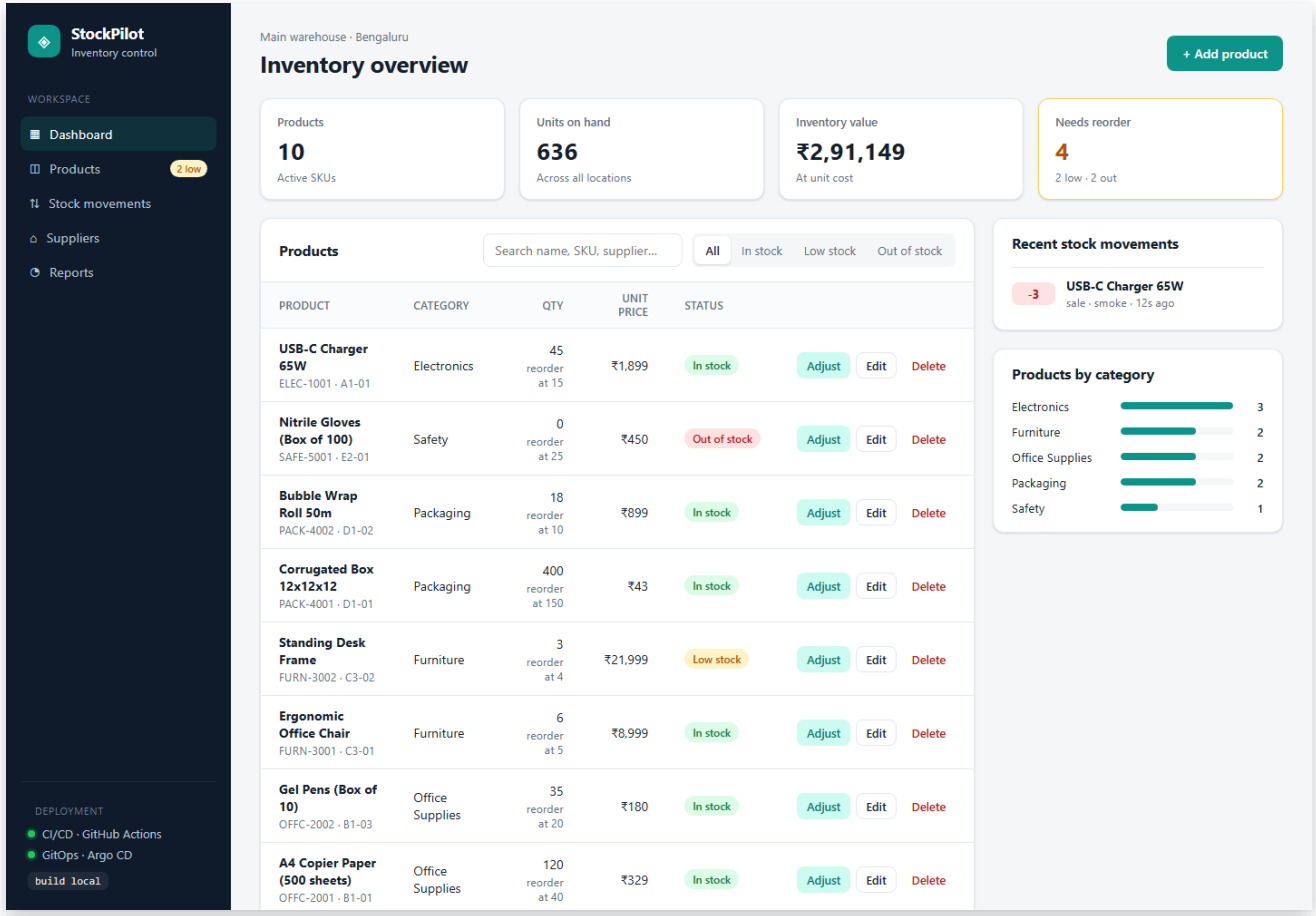
Small warehouses track stock in spreadsheets, so reorders get missed and nobody knows why a count changed.

The app

Products with SKUs, live stock levels, low/out-of-stock alerts and an audit trail of every stock movement.

The DevOps goal

Every commit is tested, scanned, deployed by GitOps to EKS and observable in Grafana, with no manual steps.



Live UI: KPI cards, searchable product table, status badges, stock-movement feed

Tech stack, layer by layer



Frontend

React 19 + Vite 8, served by unprivileged Nginx



Backend

Python 3.12, FastAPI, Pydantic v2, Uvicorn



Data

PostgreSQL 17, SQLAlchemy 2, Alembic migrations



Quality

pytest (19 tests), flake8, in-memory SQLite test DB



Containers

Multi-stage Dockerfiles, non-root, Docker Compose



CI/CD + security

GitHub Actions, Trivy gate, GHCR with SHA tags



Infrastructure

Terraform: AWS VPC, NAT, EKS, managed nodes



Platform

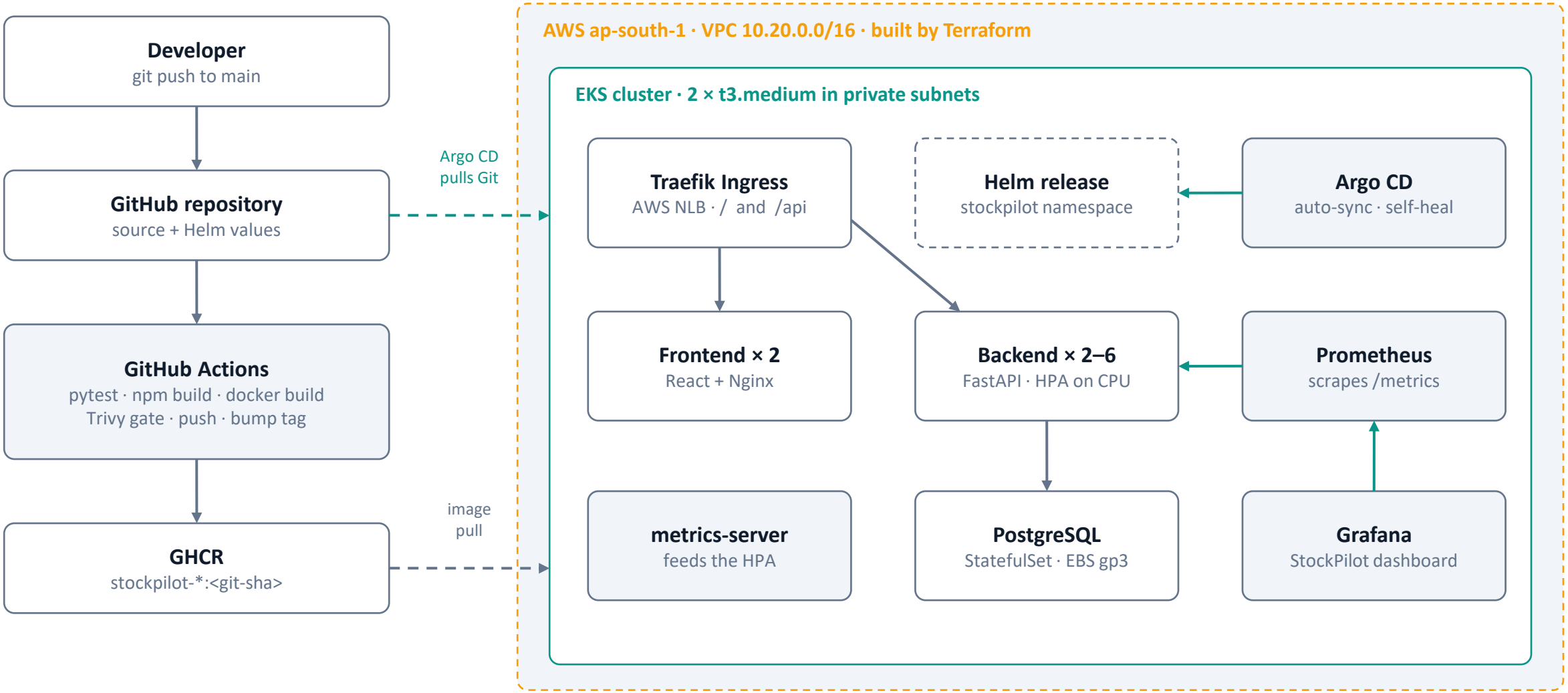
Kubernetes, Helm, Argo CD, Traefik ingress, HPA



Observability

Prometheus, ServiceMonitor, alerts, Grafana

Architecture: from git push to running pods



Application layer: REST API, database and tests

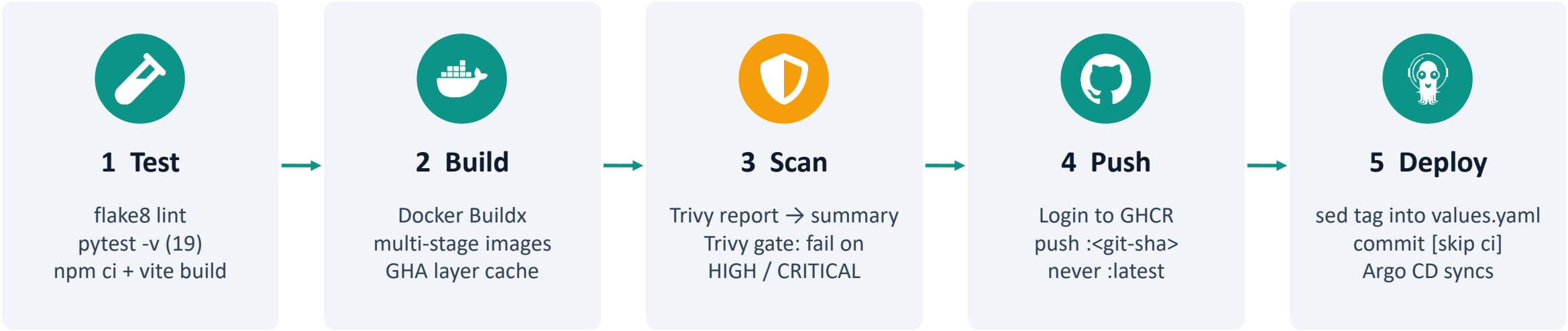
Method	Path	Purpose
GET	/health · /ready	Liveness · readiness (DB check)
GET	/metrics	Prometheus metrics
GET	/api/products	List, search, filter by status
GET	/api/products/stats	Inventory KPIs
GET	/api/products/{id}	One product
POST	/api/products	Create (409 on duplicate SKU)
PUT	/api/products/{id}	Update product
DELETE	/api/products/{id}	Delete product + history
POST	/api/products/{id}/adjust	Record stock movement
GET	/api/movements	Recent movements feed

12 HTTP endpoints, including full CRUD

2 tables via 2 Alembic migrations (schema + seed)

19 pytest cases on an isolated SQLite DB

CI/CD: every push to main runs the whole chain



Traceability: image tag = full commit SHA, e.g. ghcr.io/sidantxdev/stockpilot-backend:3f9c2e1…, also shown in the UI sidebar.

Safety: pull requests build and scan but never push. Jobs get least-privilege GITHUB_TOKEN scopes. The Trivy action is pinned by commit SHA.

No cluster credentials in CI: the pipeline only changes Git; Argo CD performs the deployment.

DevSecOps: the Trivy gate caught real CVEs

4 → 0
HIGH CVEs found on the first scan, then fixed

0
fixable HIGH / CRITICAL in either image today

0
vulnerabilities of any severity in the Alpine frontend

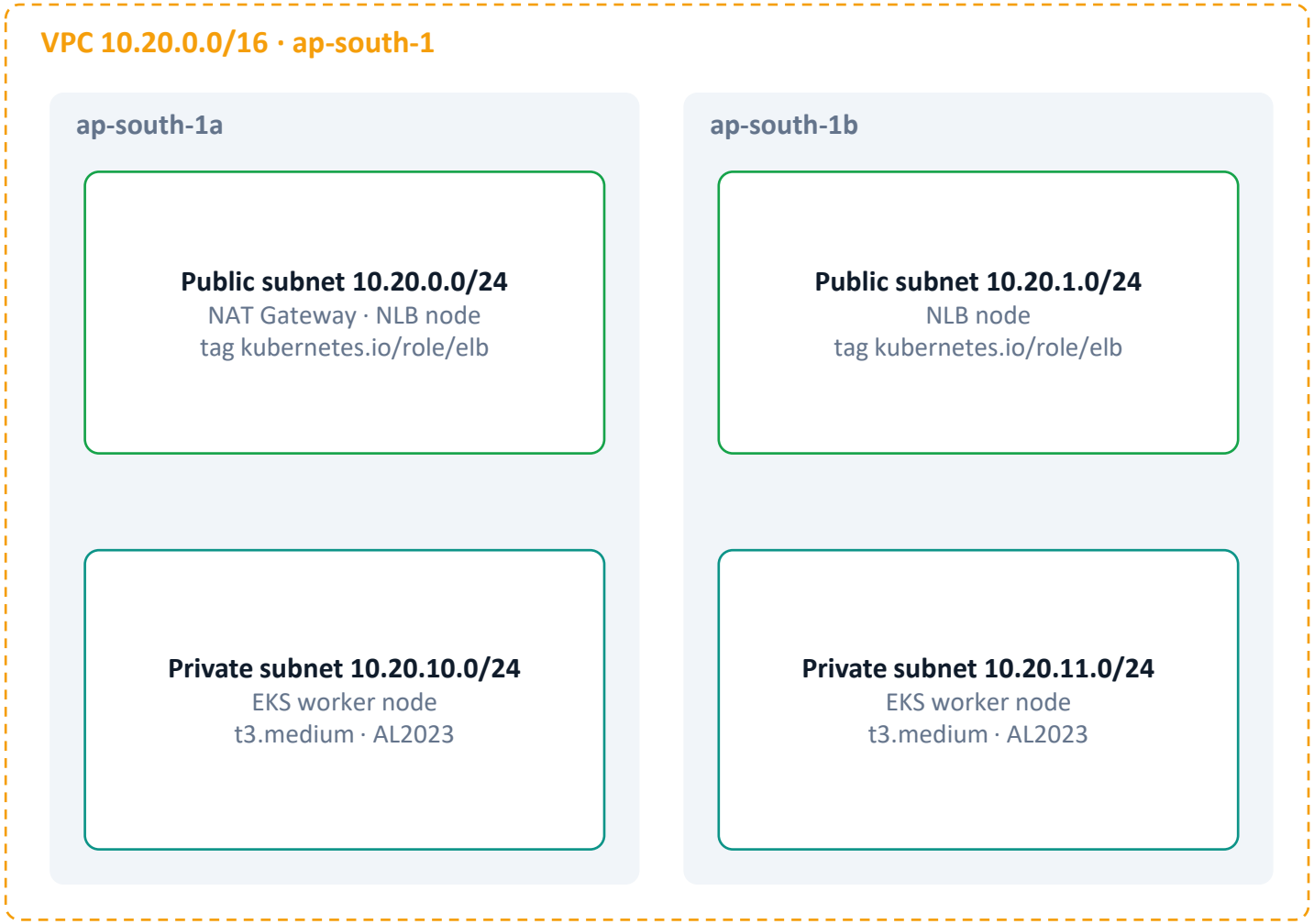
What the scan found

- urllib3 CVE-2026-97687 (TLS bypass via HTTPS proxy), CVE-2026-97689, msgpack and setuptools were flagged HIGH.
- None were app dependencies: they were vendored inside pip, which the venv carried into the runtime image.
- Fix: uninstall pip/setuptools after installing requirements. The rescan passed.
- Remaining Debian CVEs (e.g. libacl1 CVE-2026-54369) have no fix yet, so they are reported but not blocking.

Defense in depth

non-root uid 10001 / 101	**read-only root FS**	**drop ALL capabilities**
Pod Security: restricted	**no secrets in Git**	**IRSA, not node IAM**

Infrastructure as Code: Terraform builds VPC + EKS



Terraform files

- main.tf: VPC, EKS, node group, add-ons
- variables.tf / outputs.tf: inputs, kubectl command
- IRSA role for the EBS CSI driver (Postgres volume)
- terraform.tfvars.example: no credentials committed

```
terraform init → plan → apply
aws eks update-kubeconfig
terraform destroy
```

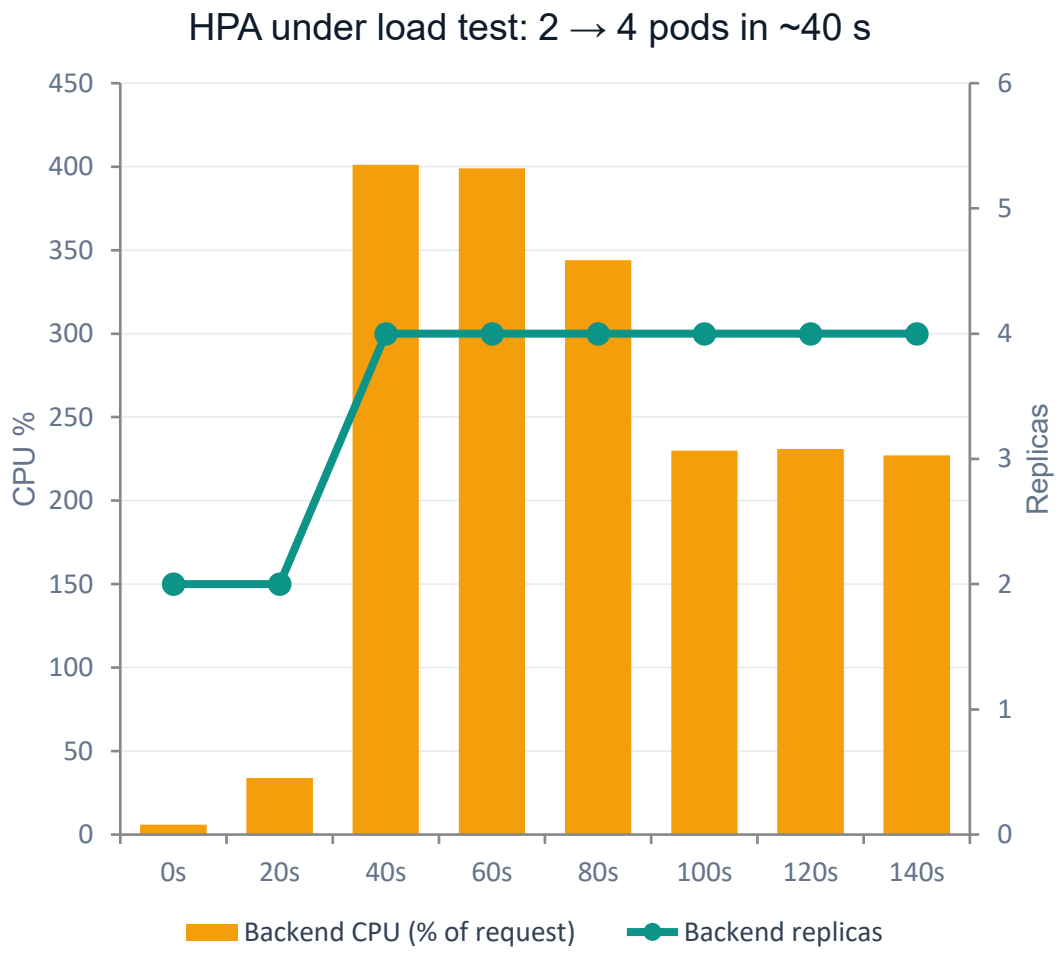
≈ $0.30 / hour

EKS + 2 nodes + NAT + NLB. Destroyed the same day.

Kubernetes + Helm: one chart, autoscaling backend

Deployment backend · 2–6 pods	**Deployment** frontend · 2 pods	**StatefulSet** PostgreSQL + PVC
Service ×3 ClusterIP	**Ingress** / → FE · /api → BE	**HPA** CPU 60% target
PDB minAvailable 1	**ServiceMonitor** Prometheus scrape	**PrometheusRule** 3 alerts
ConfigMap Grafana dashboard	**Secret + ConfigMap** DB settings	**helm test** API smoke pod

Probes: startup + liveness /health, readiness /ready. The init container waits for Postgres.



GitOps with Argo CD: Git is the source of truth



Why GitOps

- CI never holds cluster credentials
- git revert = rollback
- every change is reviewed in Git

Application spec

- auto-sync with prune + self-heal
- values.yaml + values-eks.yaml
- manual kubectl edits are reverted

Observability: Prometheus scrapes, Grafana explains



14 / 14
Prometheus targets UP

19
Grafana panels

3
alert rules

Verified on a local kind cluster with the same charts and values as EKS.

What the dashboard answers

- Is traffic arriving? Request rate by endpoint
- Is it failing? 4xx / 5xx rate
- Is it slow? p50 / p95 latency
- Is the business moving? Stock movements by reason, units in / out
- Is it scaling? Pod CPU, HPA current vs desired

Alerts: BackendDown · HighErrorRate > 5% · SlowRequests p95 > 500 ms

Key learnings

Gates before artifacts

Tests and Trivy run before the push, so a bad image never reaches the registry.

SHA tags + GitOps

Every pod traces to a commit; rollback is a git revert.

Liveness ≠ readiness

/health keeps busy pods alive; /ready keeps DB-less pods out of the load balancer.

Least privilege

Non-root, read-only FS, restricted PSS, scoped tokens, IRSA, no keys in Git.

K8s creates cloud resources

NLBs and EBS volumes live outside Terraform state, so tear-down order matters.

Measure the business too

Latency says it's healthy; stock movements say it's being used.

github.com/SidantxDev/stockpilot-devops-capstone · Thank you